

# 字符串

Morning Flower and Evening Oath

XCPC vector 模板分册

# 目录

|                          |           |
|--------------------------|-----------|
| 使用约定                     | ii        |
| <b>第一章 前缀函数与 KMP 算法</b>  | <b>1</b>  |
| 1.1 前缀函数                 | 1         |
| 1.2 KMP 匹配               | 1         |
| <b>第二章 Z 函数 (扩展 KMP)</b> | <b>3</b>  |
| 2.1 Z 函数                 | 3         |
| <b>第三章 AC 自动机</b>        | <b>4</b>  |
| 3.1 AC 自动机出现次数           | 4         |
| 3.2 AC 自动机最短后缀匹配长度       | 6         |
| <b>第四章 后缀数组 (SA)</b>     | <b>8</b>  |
| 4.1 后缀数组简介               | 8         |
| 4.1.1 倍增后缀数组与 LCP        | 8         |
| 4.1.2 后缀 LCP 与子串比较       | 11        |
| 4.1.3 两个前缀的最长公共后缀        | 12        |
| 4.1.4 重复子串 AA 的起止位置计数    | 12        |
| <b>第五章 后缀自动机 (SAM)</b>   | <b>13</b> |
| 5.1 后缀自动机                | 13        |
| <b>第六章 广义后缀自动机</b>       | <b>17</b> |
| 6.1 广义后缀自动机: Trie BFS 构建 | 17        |
| <b>第七章 Manacher</b>      | <b>21</b> |
| 7.1 Manacher 回文半径        | 21        |
| <b>第八章 回文树</b>           | <b>23</b> |
| 8.1 回文树 PAM: 回文后缀与出现次数   | 23        |
| <b>第九章 最小表示法</b>         | <b>26</b> |
| 9.1 最小循环表示               | 26        |
| <b>第十章 配套知识与验证范围</b>     | <b>27</b> |
| 10.1 回文半径与在线回文计数         | 27        |
| 10.2 整数后缀数组的边界与 SAM 次数   | 27        |
| 10.3 广义 SAM 点数与后缀语言      | 27        |

# 使用约定

C++20，默认包含 `bits/stdc++.h` 与 `cassert`，并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

# 第一章 前缀函数与 KMP 算法

## 1.1 前缀函数

`prefix_function(s)` 返回每个前缀的最长真 border 长度，0-based；空串返回空数组。O(n)，无状态，直接抄写函数。

```
1 vector<int> prefix_function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> p(n);
5     for (int i = 1; i < n; i++)
6     {
7         int j = p[i - 1];
8         while (j && s[i] != s[j]) j = p[j - 1];
9         p[i] = j + (s[i] == s[j]);
10    }
11    return p;
12 }
```

联合使用示例见第 2 页 (Luogu P3375)，其中直接调用本模板。

## 1.2 KMP 匹配

`kmp_match(s,t)` 返回非空模式串 `t` 在 `s` 中的所有起点 (0-based)，允许重叠。需要同章前缀函数。O(|s|+|t|)。

```
1 vector<int> kmp_match(const string &s, const string &t)
2 {
3     assert(!t.empty());
4     auto p = prefix_function(t);
5     vector<int> ans;
6     for (int i = 0, j = 0; i < (int)s.size(); i++)
7     {
8         while (j && s[i] != t[j]) j = p[j - 1];
9         j += (s[i] == t[j]);
10        if (j == (int)t.size())
11        {
12            ans.push_back(i - j + 1);
13            j = p[j - 1];
14        }
15    }
16    return ans;
17 }
```

## 模板题使用：Luogu P3375

题意：求模式串  $t$  在文本  $s$  中所有出现的位置，包含重叠匹配；模板返回 0-based 位置，输出加 1。最后一行直接输出 `prefix_function(t)`，它是模式串各前缀的最长真 border 长度。题目输入为非空大写串，函数按字符相等比较，不限于小写。

所需模板：`prefix_function`、`kmp_match`。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3375>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s, t;
6     cin >> s >> t;
7     for (int p : kmp_match(s, t)) cout << p + 1 << '\n';
8     for (int x : prefix_function(t)) cout << x << ' ';
9     cout << '\n';
10 }
```

## 第二章 Z 函数 (扩展 KMP)

### 2.1 Z 函数

`z_function(s)` 返回后缀 `s[i..]` 与原串的 LCP, 约定 `z[0]=n`, 空串返回空数组。O(n)。

```
1 vector<int> z_function(const string &s)
2 {
3     int n = (int)s.size();
4     vector<int> a(n);
5     if (n) a[0] = n;
6     for (int i = 1, l = 0, r = 0; i < n; i++)
7     {
8         if (i < r) a[i] = min(r - i, a[i - l]);
9         while (i + a[i] < n && s[a[i]] == s[i + a[i]]) ++a[i];
10        if (i + a[i] > r)
11        {
12            l = i;
13            r = i + a[i];
14        }
15    }
16    return a;
17 }
```

#### 模板题使用: Library Checker zalgorithm

题意: 给非空字符串, 对每个起点求其后缀与原串的最长公共前缀长度。直接输出 Z 数组, `z[0]` 等于串长。

所需模板: `z_function`。以下仅含使用代码, 默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目: <https://judge.yosupo.jp/problem/zalgorithm>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     cin >> s;
7     auto z = z_function(s);
8     for (int i = 0; i < (int)z.size(); i++)
9         cout << z[i] << (i + 1 == (int)z.size() ? '\n' : ' ');
10 }
```

# 第三章 AC 自动机

## 3.1 AC 自动机出现次数

字母表  $a..z$ ,  $\text{add}(s)$  返回终止状态, 要求模式非空; 全部插入后  $\text{build}()$ , 只允许一次构建。 $\text{count}(\text{text})$  返回每个状态对应模式的出现次数, 可重复查询; 重复模式共享状态。构建  $O(26N)$ , 查询  $O(|\text{text}|+N)$ 。

```
1 struct AhoCorasick
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int fail = 0;
7     };
8
9     vector<Node> a{Node{}};
10    vector<int> order;
11    bool built = false;
12
13    int add(const string &s)
14    {
15        assert(!built && !s.empty());
16        int u = 0;
17        for (char ch : s)
18        {
19            int c = ch - 'a';
20            assert(0 <= c && c < 26);
21            if (!a[u].go[c])
22            {
23                int v = (int)a.size();
24                a[u].go[c] = v;
25                a.push_back(Node{});
26            }
27            u = a[u].go[c];
28        }
29        return u;
30    }
31
32    void build()
33    {
34        assert(!built);
35        built = true;
36        queue<int> q;
37        for (int v : a[0].go)
38            if (v) q.push(v);
39        while (!q.empty())
40        {
41            int u = q.front();
42            q.pop();
43            order.push_back(u);
44            for (int c = 0; c < 26; c++)
45            {
```

```
46         int v = a[u].go[c];
47         if (v)
48         {
49             a[v].fail = a[a[u].fail].go[c];
50             q.push(v);
51         }
52         else
53             a[u].go[c] = a[a[u].fail].go[c];
54     }
55 }
56
57
58 vector<long long> count(const string &s) const
59 {
60     assert(built);
61     vector<long long> ans(a.size());
62     int u = 0;
63     for (char ch : s)
64     {
65         u = a[u].go[ch - 'a'];
66         ++ans[u];
67     }
68     for (int i = (int)order.size() - 1; i >= 0; i--)
69         ans[a[order[i]].fail] += ans[order[i]];
70     return ans;
71 }
72 };
```

## 模板题使用：Luogu P5357

题意：给多个小写字母模式和文本，输出每个模式的出现次数，允许重叠。先保存 add 返回的终止状态，全部插入后 build，再 count 文本并按状态取值；重复模式共享状态，仍按输入顺序输出。

所需模板：AhoCorasick。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P5357>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     AhoCorasick ac;
8     vector<int> terminal(n);
9     for (int &id : terminal)
10    {
11        string pattern;
12        cin >> pattern;
13        id = ac.add(pattern);
14    }
15    ac.build();
16    string text;
17    cin >> text;
18    auto count = ac.count(text);
19    for (int id : terminal) cout << count[id] << '\n';
20 }
```

## 3.2 AC 自动机最短后缀匹配长度

先用 AhoCorasick.add(s) 得到终止节点，记录 (节点, 模式长度)，再 build 后调用 ac\_shortest(ac, ends)。返回每个状态沿 fail 链对应的最短已登记模式长度，无模式为 INT\_MAX。登记长度须真实且在 1..INT\_MAX-1；重复模式、部分模式集合、空集合均可，未登记的模式不会参与最小值。按 ac.order 的 BFS 顺序传播，父 fail 状态先完成；O(状态数 + 登记数) 时间，O(状态数) 额外空间。扫描文本时 u=ac.a[u].go[c]，len[u] 就是当前位置结尾的最短匹配长度；文本仍须为小写字母。辅助函数不修改自动机，也不影响 count(text)，可对同一自动机的不同登记集合重复调用。

```

1 vector<int> ac_shortest(const AhoCorasick &ac, const vector<pair<int, int>> &ends)
2 {
3     assert(ac.built);
4     vector<int> len(ac.a.size(), INT_MAX);
5     for (auto [u, n] : ends)
6     {
7         assert(0 < u && u < (int)len.size() && 0 < n && n < INT_MAX);
8         len[u] = min(len[u], n);
9     }
10    for (int u : ac.order) len[u] = min(len[u], len[ac.a[u].fail]);
11    return len;
12 }
```

## 应用题补充：AC 最短后缀匹配（本地接口示例）

题意：输入模式数、各个非空小写模式串及文本，逐位置输出以该位置结尾的最短模式长度，无匹配输出 0。先登记终止节点与长度，再 build 和传播。示例对应附件 min\_len 变体，只作本地接口演示，不是 OJ 提交记录。

所需模板：AhoCorasick、ac\_shortest。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://oi-wiki.org/string/ac-automaton/>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     AhoCorasick ac;
8     vector<pair<int, int>> ends;
9     while (n--)
10    {
11        string s;
12        cin >> s;
13        ends.push_back({ac.add(s), (int)s.size()});
14    }
15    ac.build();
16    auto len = ac_shortest(ac, ends);
17    string s;
18    cin >> s;
19    int u = 0;
20    for (char c : s)
21    {
22        u = ac.a[u].go[c - 'a'];
23        cout << (len[u] == INT_MAX ? 0 : len[u]) << ' ';
24    }
25    cout << '\n';
26 }
```

# 第四章 后缀数组 (SA)

## 4.1 后缀数组简介

### 4.1.1 倍增后缀数组与 LCP

构造 SuffixArray(s) 支持任意字节串, 按 unsigned char 的 0..255 数值比较, 包含内嵌零字节; 也可构造 SuffixArray(a,alphabet), 整数序列要求  $0 \leq a[i] < \text{alphabet}$ ,  $1 \leq \text{alphabet} < \text{INT\_MAX}$ 。重新赋值 SuffixArray(a,alphabet) 可重建。长度须小于 INT\_MAX。sa[k] 为第 k 个非空后缀起点, rk[i] 为排名, 全部 0-based; lcp[k] 为 sa[k-1] 与 sa[k] 的最长公共前缀, lcp[0]=0。空序列合法, 三数组均为空。无需手动追加哨兵, 也不输出空后缀。每轮按第二关键字已有顺序、第一关键字稳定计数排序, 倍增  $O(n \log n + \text{alphabet})$ , 空间  $O(n + \text{alphabet})$ ; 字节串 alphabet=256。大整数先自行保序离散化, 避免按最大原值开桶。Kasai 求 lcp 为  $O(n)$ 。不同起点 x,y 的 LCP 为  $\text{lcp}[\min(\text{rk}[x], \text{rk}[y]) + 1.. \max(\text{rk}[x], \text{rk}[y])]$  的最小值; 本结构不内建 RMQ, 需要频繁查询时另接静态 RMQ。同一起点的 LCP 为 n-x。覆盖倍增构造, 不替代 DC3/SA-IS。

```
1 struct SuffixArray
2 {
3     vector<int> sa, rk, lcp;
4
5     SuffixArray(const string &s)
6     {
7         vector<int> a;
8         a.reserve(s.size());
9         for (unsigned char c : s) a.push_back(c);
10        *this = SuffixArray(a, 256);
11    }
12
13    SuffixArray(const vector<int> &s, int alphabet)
14    {
15        assert(s.size() < INT_MAX);
16        assert(1 <= alphabet && alphabet < INT_MAX);
17        int n = (int)s.size();
18        sa.resize(n);
19        rk.resize(n);
20        lcp.assign(n, 0);
21        if (!n) return;
22        vector<int> order(n), old(n), cnt(max(n + 1, alphabet + 1));
23        int m = alphabet;
24        for (int i = 0; i < n; i++)
25        {
26            assert(0 <= s[i] && s[i] < alphabet);
27            rk[i] = s[i] + 1;
28            cnt[rk[i]]++;
29        }
30        for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
31        for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[i]]] = i;
```

```

32     for (int k = 1; k < n; k *= 2)
33     {
34         int p = 0;
35         for (int i = n - k; i < n; i++) order[p++] = i;
36         for (int i : sa)
37             if (i >= k) order[p++] = i - k;
38         fill(cnt.begin(), cnt.begin() + m + 1, 0);
39         for (int x : rk) cnt[x]++;
40         for (int i = 1; i <= m; i++) cnt[i] += cnt[i - 1];
41         for (int i = n - 1; i >= 0; i--) sa[--cnt[rk[order[i]]]] = order[i];
42         old.swap(rk);
43         auto second = [&](int i)
44         {
45             return i < n - k ? old[i + k] : 0;
46         };
47         m = 1;
48         rk[sa[0]] = m;
49         for (int i = 1; i < n; i++)
50         {
51             int a = sa[i - 1], b = sa[i];
52             if (old[a] != old[b] || second(a) != second(b)) m++;
53             rk[b] = m;
54         }
55         if (m == n) break;
56     }
57     for (int i = 0; i < n; i++) rk[sa[i]] = i;
58     for (int i = 0, k = 0; i < n; i++)
59     {
60         if (!rk[i])
61         {
62             k = 0;
63             continue;
64         }
65         int j = sa[rk[i] - 1];
66         while (k < n - i && k < n - j && s[i + k] == s[j + k]) ++k;
67         lcp[rk[i]] = k;
68         if (k) --k;
69     }
70 }
71 };

```

## 模板题使用：Luogu P3809

题意：把所有非空后缀按字典序排序，输出各后缀在原串中的起点。模板 sa 为 0-based，题目输出为 1-based，因此加 1。

所需模板：SuffixArray。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3809>

```

1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     std::string s;
6     std::cin >> s;
7     SuffixArray suffix(s);
8     for (int i = 0; i < (int)s.size(); i++)
9         std::cout << suffix.sa[i] + 1 << (i + 1 == (int)s.size() ? '\n' : ' ');
10 }
```

## 模板题使用：Library Checker longest\_common\_substring

题意：求两个小写字字符串的一段最长公共子串，输出两串各自 0-based 半开区间，各串长度至 50 万。整数拼接以 26 为分隔符、alphabet=27；扫描异来源相邻后缀，并将 LCP 截断至原串边界。

所需模板：SuffixArray。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：[https://judge.yosupo.jp/problem/longest\\_common\\_substring](https://judge.yosupo.jp/problem/longest_common_substring)

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s, t;
6     cin >> s >> t;
7     int n = s.size(), m = t.size();
8     vector<int> a;
9     for (char c : s) a.push_back(c - 'a');
10    a.push_back(26);
11    for (char c : t) a.push_back(c - 'a');
12    SuffixArray suffix(a, 27);
13    int best = 0, left = 0, right = 0;
14    for (int k = 1; k < (int)a.size(); k++)
15    {
16        int x = suffix.sa[k - 1], y = suffix.sa[k];
17        if (x > y) swap(x, y);
18        if (x >= n || y <= n) continue;
19        int len = min({suffix.lcp[k], n - x, n + 1 + m - y});
20        if (len > best)
21        {
22            best = len;
23            left = x;
24            right = y - n - 1;
25        }
26    }
27    cout << left << ' ' << left + best << ' ' << right << ' ' << right + best << '\n';
28 }
```

### 4.1.2 后缀 LCP 与子串比较

先构造 SuffixArray，再构造 SuffixLCP(sa)。重新赋值 SuffixLCP(sa) 可建新表。复制所需排名与 LCP 数据，不保存原字符串或源对象引用，源对象可销毁或重建。query(x,y) 返回 s[x..n] 与 s[y..n] 的最长公共前缀， $0 \leq x, y \leq n$ ，允许空后缀；同起点返回 n-x。compare(l1,r1,l2,r2) 比较两个 0-based 半开子串，允许空串，返回 -1/0/1。若公共前缀覆盖较短子串，按长度比较，否则原后缀排名给出首个不同字符的顺序。字节排序及整数字母表约束继承 SuffixArray。预处理稀疏表  $O(n \log n)$  时间、空间，每次查询  $O(1)$ ，无需哈希和原串。n、rk、lg、st 可读，禁止调用方破坏。

```

1 struct SuffixLCP
2 {
3     int n;
4     vector<int> rk, lg;
5     vector<vector<int>> st;
6
7     SuffixLCP(const SuffixArray &suffix)
8     {
9         n = suffix.sa.size();
10        rk = suffix.rk;
11        lg.assign(n + 1, 0);
12        for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
13        st.assign(lg[n] + 1, {});
14        st[0] = suffix.lcp;
15        for (int k = 1; k <= lg[n]; k++)
16        {
17            int len = 1 << k;
18            st[k].resize(n - len + 1);
19            for (int i = 0; i <= n - len; i++)
20                st[k][i] = min(st[k - 1][i], st[k - 1][i + len / 2]);
21        }
22    }

```

```

24 int query(int x, int y) const
25 {
26     assert(0 <= x && x <= n && 0 <= y && y <= n);
27     if (x == y || x == n || y == n) return n - max(x, y);
28     int a = rk[x], b = rk[y];
29     if (a > b) swap(a, b);
30     int k = lg[b - a];
31     return min(st[k][a + 1], st[k][b - (1 << k) + 1]);
32 }
33
34 // Compare [l1,r1) and [l2,r2); return -1, 0 or 1.
35 int compare(int l1, int r1, int l2, int r2) const
36 {
37     assert(0 <= l1 && l1 <= r1 && r1 <= n);
38     assert(0 <= l2 && l2 <= r2 && r2 <= n);
39     int a = r1 - l1, b = r2 - l2;
40     if (query(l1, l2) >= min(a, b)) return (a > b) - (a < b);
41     return rk[l1] < rk[l2] ? -1 : 1;
42 }
43 };

```

### 4.1.3 两个前缀的最长公共后缀

`prefix_lcs(reversed,x,y)` 返回原序列 `s[0..x)` 与 `s[0..y)` 的最长公共后缀，端点满足  $0 \leq x, y \leq n$ 。reversed 必须在原序列的反转序列上构造 SuffixLCP，不能传正向对象；转换为 `reversed.query(n-x,n-y)`，单次  $O(1)$ 。只需前向 LCP 时不必构造反向对象；该接口中的 LCS 是最长公共后缀，不是最长公共子序列。对于两个任意子串，以右端点查询后还需截断到两个子串长度的最小值。

```
1 // reversed must be built on the reversed original sequence.
2 int prefix_lcs(const SuffixLCP &reversed, int x, int y)
3 {
4     assert(0 <= x && x <= reversed.n && 0 <= y && y <= reversed.n);
5     return reversed.query(reversed.n - x, reversed.n - y);
6 }
```

### 4.1.4 重复子串 AA 的起止位置计数

`square_counts(forward,reversed)` 返回 `(start,finish)` 两个长度 `n` 的 long long 数组，`start[i]/finish[i]` 为开始/结束于位置 `i` 的非空 AA 型子串个数；结束位置是最后一个字符下标，均 0-based。两对象必须分别对应同一原序列及其反序列，长度相同，允许空序列。枚举半长 `len`，并以其整数倍 `i` 为锚点；向左匹配截断为 `len-1`，向右截断为 `len`，由 LCP/LCS 得出合法起点连续区间，差分累计。左侧不能放宽到 `len`，否则相邻锚点会重复计数。预处理查询器后，此函数  $O(n \log n)$  时间、 $O(n)$  额外空间。不同出现位置、不同半长分别计数；不要求 `A` 为最小周期。P1117 的 AABB 拆分数为所有分界点 `finish[i]*start[i+1]` 之和，累加类型需按题目范围选择，函数本身只返回起止计数。

```
1 // Count nonempty squares AA by their starting and ending positions.
2 pair<vector<long long>, vector<long long>> square_counts(const SuffixLCP &forward,
3                                                         const SuffixLCP &reversed)
4 {
5     int n = forward.n;
6     assert(reversed.n == n);
7     vector<long long> start(n + 1), finish(n + 1);
8     for (int len = 1; len <= n / 2; len++)
9         for (int i = 0; i < n - len; i += len)
10            {
11                int j = i + len;
12                int left = min(len - 1, prefix_lcs(reversed, i, j));
13                int right = min(len, forward.query(i, j));
14                if (left + right < len) continue;
15                int l = i - left, r = i + right - len;
16                start[l]++;
17                start[r + 1]--;
18                finish[l + 2 * len - 1]++;
19                finish[r + 2 * len]--;
20            }
21     for (int i = 1; i < n; i++)
22     {
23         start[i] += start[i - 1];
24         finish[i] += finish[i - 1];
25     }
26     start.resize(n);
27     finish.resize(n);
28     return {start, finish};
29 }
```

# 第五章 后缀自动机 (SAM)

## 5.1 后缀自动机

extend(c) 接收整数 0..25, 每次追加一个小写字符; last 为整串状态, a[0] 为根, 根 link=-1。a[u].len 为状态最大长度, 非根代表 (len[link[u]],len[u]) 且具有同一 endpos 集合。occ 是原始种子: 真实扩展为 1、克隆为 0, 不能直接当出现次数。counts() 按长度倒序汇总后返回副本, 不改结构, 重复查询及查询后继续追加安全; 根计数为 n, 非空子串只看 u>0。distinct() 为不同非空子串数。重赋值 SuffixAutomaton() 清空; 固定 26 字母表构建 O(n), counts/distinct 为 O(n), 空间 O(26n)。如需前缀位置对应状态, 在每次 extend 后保存 last; 本结构不输出广义 SAM 或最小 DFA 状态数。

```
1 struct SuffixAutomaton
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int link = -1, len = 0;
7         long long occ = 0;
8     };
9
10    vector<Node> a{Node{}};
11    int last = 0;
```

接上页同一结构体：

```
13 void extend(int c)
14 {
15     assert(0 <= c && c < 26);
16     int cur = (int)a.size();
17     a.push_back(Node{});
18     a[cur].len = a[last].len + 1;
19     a[cur].occ = 1;
20     int p = last;
21     while (p != -1 && !a[p].go[c])
22     {
23         a[p].go[c] = cur;
24         p = a[p].link;
25     }
26     if (p == -1)
27         a[cur].link = 0;
28     else
29     {
30         int q = a[p].go[c];
31         if (a[p].len + 1 == a[q].len)
32             a[cur].link = q;
33         else
34         {
35             int clone = (int)a.size();
36             Node copy = a[q];
37             a.push_back(copy);
38             a[clone].len = a[p].len + 1;
39             a[clone].occ = 0;
40             while (p != -1 && a[p].go[c] == q)
41             {
42                 a[p].go[c] = clone;
43                 p = a[p].link;
44             }
45             a[q].link = a[cur].link = clone;
46         }
47     }
48     last = cur;
49 }
```

接上页同一结构体：

```

51     vector<long long> counts() const
52     {
53         vector<int> bucket(a[last].len + 1), order(a.size());
54         vector<long long> ans(a.size());
55         for (auto &v : a) ++bucket[v.len];
56         partial_sum(bucket.begin(), bucket.end(), bucket.begin());
57         for (int i = 0; i < (int)a.size(); i++)
58         {
59             order[--bucket[a[i].len]] = i;
60             ans[i] = a[i].occ;
61         }
62         for (int i = (int)order.size() - 1; i > 0; i--)
63             ans[a[order[i]].link] += ans[order[i]];
64         return ans;
65     }
66
67     long long distinct() const
68     {
69         long long ans = 0;
70         for (int i = 1; i < (int)a.size(); i++) ans += a[i].len - a[a[i].link].len;
71         return ans;
72     }
73 };

```

## 模板题使用：Library Checker number\_of\_substrings

题意：给小写字母串，求不同非空子串的个数。extend 接收 0..25 的整数，故字符减去 a；所有字符追加后调用 distinct，结果使用 long long。

所需模板：SuffixAutomaton。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：[https://judge.yosupo.jp/problem/number\\_of\\_substrings](https://judge.yosupo.jp/problem/number_of_substrings)

```

1  int main()
2  {
3      ios::sync_with_stdio(false);
4      cin.tie(nullptr);
5      string s;
6      cin >> s;
7      SuffixAutomaton sam;
8      for (char c : s) sam.extend(c - 'a');
9      cout << sam.distinct() << '\n';
10 }

```

## 模板题使用：Luogu P3804

题意：长度至 100 万的小写串，只考虑出现次数大于 1 的子串，最大化长度乘次数。counts 按长度汇总，取非根状态  $\text{len} * \text{count}$ ；无重复子串输出 0，乘积使用 long long。

所需模板：SuffixAutomaton。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3804>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     cin >> s;
7     SuffixAutomaton sam;
8     for (char c : s) sam.extend(c - 'a');
9     auto count = sam.counts();
10    long long answer = 0;
11    for (int u = 1; u < (int)sam.a.size(); u++)
12    {
13        if (count[u] > 1) answer = max(answer, count[u] * sam.a[u].len);
14    }
15    cout << answer << '\n';
16 }
```

# 第六章 广义后缀自动机

## 6.1 广义后缀自动机：Trie BFS 构建

先 `add(s)` 登记若干小写字符串，再 `build()` 原地将 Trie 变为广义 SAM；`build` 后禁止继续 `add`，可重复 `build`，重新收集串需新建对象。空串与重复串允许，`distinct()` 返回所有输入串中不同非空子串的总数，不包含跨串拼接。根为 0，`a[v].go[c]` 为转移、0 表示无边，`link` 为后缀链接、`len` 为最长代表串长度；从根沿字符转移可判定子串是否存在。`a.size()` 为按文档 `endpos` 等价类构造的状态数，不应直接称为后缀并集语言的最小 DFA 点数；`ab`、`b` 是两者不同的例子。P6139 官方样例一要求 `endpos` 点数 10，后缀语言最小化仅 7 点；该题用 `a.size()`，勿再最小化。固定 26 字符，设累计输入长度 `L`、Trie 点数 `K`，登记总时间  $O(L)$ ，`build` 时间与空间  $O(26K)$ ，状态少于  $2K$ ；累计输入长度及  $2K$  须可由 `int` 表示。克隆只复制已处理转移，不能带入尚未 BFS 处理的 Trie 子边。此版不提供出现次数或各串所属标记。

```
1 struct GeneralSAM
2 {
3     struct Node
4     {
5         array<int, 26> go{};
6         int link = -1, len = 0;
7     };
8
9     vector<Node> a{Node{}};
10    bool ready = false;
11
12    void add(const string &s)
13    {
14        assert(!ready);
15        int p = 0;
16        for (char ch : s)
17        {
18            int c = ch - 'a';
19            assert(0 <= c && c < 26);
20            if (!a[p].go[c])
21            {
22                int v = a.size();
23                a.push_back(Node{});
24                a[p].go[c] = v;
25            }
26            p = a[p].go[c];
27        }
28    }
```

BFS 构建（接上页同一结构体）：

```

30 void build()
31 {
32     if (ready) return;
33     a.reserve(2 * a.size());
34     queue<pair<int, int>> todo;
35     for (int c = 0; c < 26; c++)
36         if (a[0].go[c]) todo.push({0, c});
37     while (!todo.empty())
38     {
39         auto [parent, c] = todo.front();
40         todo.pop();
41         int cur = a[parent].go[c], p = a[parent].link;
42         a[cur].len = a[parent].len + 1;
43         while (p != -1 && !a[p].go[c])
44         {
45             a[p].go[c] = cur;
46             p = a[p].link;
47         }
48         if (p == -1)
49             a[cur].link = 0;
50         else
51         {
52             int q = a[p].go[c];
53             if (a[p].len + 1 == a[q].len)
54                 a[cur].link = q;
55             else
56             {
57                 int clone = a.size();
58                 Node copy = a[q];
59                 copy.len = a[p].len + 1;
60                 for (int &v : copy.go)
61                     if (!a[v].len) v = 0;
62                 a.push_back(copy);
63                 while (p != -1 && a[p].go[c] == q)
64                 {
65                     a[p].go[c] = clone;
66                     p = a[p].link;
67                 }
68                 a[q].link = a[cur].link = clone;
69             }
70         }
71         for (int ch = 0; ch < 26; ch++)
72             if (int v = a[cur].go[ch]; v && !a[v].len) todo.push({cur, ch});
73     }
74     ready = true;
75 }

```

不同子串计数（接上页同一结构体）：

```
77     long long distinct() const
78     {
79         assert(ready);
80         long long ans = 0;
81         for (int v = 1; v < (int)a.size(); v++) ans += a[v].len - a[a[v].link].len;
82         return ans;
83     }
84 };
```

## 模板题使用：Library Checker number\_of\_substrings

题意：本题只有一个字符串，输出不同非空子串数。广义模板可在 build 前多次 add；本例验证其单串特例，多串行为另有独立对拍。若只处理单串，原 SuffixAutomaton 更合适。此题不验证广义 SAM 状态数或最小 DFA 状态数。

所需模板：GeneralSAM。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：[https://judge.yosupo.jp/problem/number\\_of\\_substrings](https://judge.yosupo.jp/problem/number_of_substrings)

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     cin >> s;
7     GeneralSAM sam;
8     sam.add(s);
9     sam.build();
10    cout << sam.distinct() << '\n';
11 }
```

## 模板题使用：Luogu P6139

题意：读入所有小写字符串，求不同非空子串数及含根的广义 SAM 点数。先全部 add，再 build，输出 distinct 和 a.size；总长度至 100 万、文档数至 40 万。点数按官方样例的 endpos 等价类口径：aa、ab、bac、caa 输出 10 和 10，不另做后缀语言 DFA 最小化。本例通过完整驱动本地验证，线上待补。

所需模板：GeneralSAM。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P6139>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     cin >> n;
7     GeneralSAM sam;
8     while (n--)
9     {
10         string s;
11         cin >> s;
12         sam.add(s);
13     }
14     sam.build();
15     cout << sam.distinct() << '\n' << sam.a.size() << '\n';
16 }
```

# 第七章 Manacher

## 7.1 Manacher 回文半径

manacher(s) 返回 odd/even, 下标从 0 开始。odd[i] 包含中心, 闭区间 [i-odd[i]+1,i+odd[i]-1], 长度 2\*odd[i]-1; even[i] 的中心在 i 前, 区间 [i-even[i],i+even[i]-1], 长度 2\*even[i], 零半径为空。空串合法, 支持任意字节 (含零), 按字节而非 Unicode 字素比较; int 索引要求  $n < 2^{30}$ 。最长回文取上述长度最大值; 全部 odd/even 以 long long 求和得到回文出现总数。O(n) 时间和空间, 边界显式检查, 无需选择哨兵字符。

```
1 // odd[i] includes the center; even[i] is centered before i.
2 pair<vector<int>, vector<int>> manacher(const string &s)
3 {
4     int n = (int)s.size();
5     vector<int> odd(n), even(n);
6     for (int i = 0, l = 0, r = -1; i < n; i++)
7     {
8         int k = i > r ? 1 : min(odd[l + r - i], r - i + 1);
9         while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k;
10        odd[i] = k--;
11        if (i + k > r)
12        {
13            l = i - k;
14            r = i + k;
15        }
16    }
17    for (int i = 0, l = 0, r = -1; i < n; i++)
18    {
19        int k = i > r ? 0 : min(even[l + r - i + 1], r - i + 1);
20        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k;
21        even[i] = k--;
22        if (i + k > r)
23        {
24            l = i - k - 1;
25            r = i + k;
26        }
27    }
28    return {odd, even};
29 }
```

## 模板题使用：Library Checker enumerate\_palindromes

题意：给非空字符串，从左到右输出每个字符及相邻字符间隙处的最长回文长度，共  $2n-1$  项。odd 含中心，even[i] 的中心在 i 前面，因此分别转成  $2*odd[i]-1$  和  $2*even[i]$ 。

所需模板：manacher。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：[https://judge.yosupo.jp/problem/enumerate\\_palindromes](https://judge.yosupo.jp/problem/enumerate_palindromes)

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     cin >> s;
7     int n = s.size();
8     auto [odd, even] = manacher(s);
9     vector<int> answer(2 * n - 1);
10    for (int i = 0; i < n; i++)
11    {
12        answer[2 * i] = 2 * odd[i] - 1;
13        if (i) answer[2 * i - 1] = 2 * even[i];
14    }
15    for (int i = 0; i < (int)answer.size(); i++)
16        cout << answer[i] << (i + 1 == (int)answer.size() ? '\n' : ' ');
17 }
```

## 模板题使用：Luogu P3805

题意：给定长度不超过 11000000 的小写字母串，输出最长回文子串长度。odd 含中心，even[i] 以 i 前间隙为中心，分别用  $2*odd[i]-1$  和  $2*even[i]$  取最大值。

所需模板：manacher。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3805>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     string s;
6     cin >> s;
7     auto [odd, even] = manacher(s);
8     int answer = 0;
9     for (int i = 0; i < (int)s.size(); i++)
10    {
11        answer = max(answer, 2 * odd[i] - 1);
12        answer = max(answer, 2 * even[i]);
13    }
14    cout << answer << '\n';
15 }
```

# 第八章 回文树

## 8.1 回文树 PAM: 回文后缀与出现次数

`add(ch)` 追加小写英文字母，返回最长回文后缀节点，亦保存于 `last`。节点 0 是长度 0 根，1 是长度 -1 根，实际回文节点从 2 开始；`distinct()` 为不同非空回文串数。`t[u].len` 为长度，`fail` 为最长真回文后缀链接，`suffixes` 为该节点的非空回文后缀数量；追加后的 `t[last].suffixes` 就是本位置结尾的回文串数。`total` 为当前全部回文子串的出现次数之和。`end` 为首次出现时的 0-based 右端点，可恢复原串子段；`hits` 仅统计作为最長后缀的次数，不能当出现次数。`occurrences()` 返回按节点编号的完整出现次数副本，根置零， $O(n)$ ；重复调用以及查询后继续追加均安全。重新赋值 `PalindromicTree()` 清空重建。固定 26 字符集下建树  $O(n)$ ，空间  $O(26n)$ 。

```
1 struct PalindromicTree
2 {
3     struct Node
4     {
5         int next[26]{};
6         int len = 0, fail = 0, suffixes = 0, end = -1;
7         long long hits = 0;
8     };
9
10    vector<Node> t;
11    vector<int> s;
12    int last = 0;
13    long long total = 0;
14
15    PalindromicTree() : t(2), s(1, -1)
16    {
17        t[0].fail = 1;
18        t[1].len = -1;
19    }
20
21    int get_fail(int u) const
22    {
23        int pos = s.size() - 1;
24        while (s[pos - t[u].len - 1] != s[pos]) u = t[u].fail;
25        return u;
26    }
```

接上页同一结构体：

```

28 // Append a lowercase letter; return the longest palindromic suffix node.
29 int add(char ch)
30 {
31     int c = ch - 'a';
32     assert(0 <= c && c < 26);
33     s.push_back(c);
34     int u = get_fail(last);
35     if (!t[u].next[c])
36     {
37         int v = t.size();
38         t.push_back(Node{});
39         t[v].len = t[u].len + 2;
40         t[v].fail = t[get_fail(t[u].fail)].next[c];
41         t[v].suffixes = t[t[v].fail].suffixes + 1;
42         t[v].end = (int)s.size() - 2;
43         t[u].next[c] = v;
44     }
45     last = t[u].next[c];
46     t[last].hits++;
47     total += t[last].suffixes;
48     return last;
49 }
50
51 int distinct() const { return (int)t.size() - 2; }
52
53 // Copy-and-propagate: repeated calls and later appends remain valid.
54 vector<long long> occurrences() const
55 {
56     vector<long long> count(t.size());
57     for (int u = 2; u < (int)t.size(); u++) count[u] = t[u].hits;
58     for (int u = (int)t.size() - 1; u >= 2; u--) count[t[u].fail] += count[u];
59     count[0] = 0;
60     count[1] = 0;
61     return count;
62 }
63 };

```

## 模板题使用：Luogu P5496

题意：按上一次答案把当前字符循环偏移后追加，输出以当前位置结尾的回文串个数。add 返回最长回文后缀节点，读取其 suffixes。

所需模板：PalindromicTree。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P5496>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     std::string s;
6     std::cin >> s;
7     PalindromicTree pam;
8     int answer = 0;
9     for (int i = 0; i < (int)s.size(); i++)
10    {
11        char ch = 'a' + (s[i] - 'a' + answer) % 26;
12        int u = pam.add(ch);
13        answer = pam.t[u].suffixes;
14        std::cout << answer << (i + 1 == (int)s.size() ? '\n' : ' ');
15    }
16 }
```

## 应用题补充：Luogu P3649

题意：求所有回文子串中“长度乘出现次数”的最大值。整串追加完后调用 occurrences 汇总次数，不能直接把 hits 当完整出现次数。

所需模板：PalindromicTree。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3649>

```
1 int main()
2 {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     std::string s;
6     std::cin >> s;
7     PalindromicTree pam;
8     for (char ch : s) pam.add(ch);
9     auto count = pam.occurrences();
10    long long answer = 0;
11    for (int u = 2; u < (int)count.size(); u++)
12        answer = std::max(answer, count[u] * pam.t[u].len);
13    std::cout << answer << '\n';
14 }
```

# 第九章 最小表示法

## 9.1 最小循环表示

`minimum_rotation(s)` 返回字典序最小循环表示的起点 (0-based), 空串返回 0; 按无符号字节比较, 与 `string` 字典序一致。O(n)。

```
1 int minimum_rotation(const string &s)
2 {
3     int n = (int)s.size(), i = 0, j = 1, k = 0;
4     if (!n) return 0;
5     while (i < n && j < n && k < n)
6     {
7         unsigned char a = s[(i + k) % n], b = s[(j + k) % n];
8         if (a == b)
9         {
10             ++k;
11             continue;
12         }
13         if (a > b)
14         {
15             i += k + 1;
16             if (i == j) ++i;
17         }
18         else
19         {
20             j += k + 1;
21             if (i == j) ++j;
22         }
23         k = 0;
24     }
25     return min(i, j);
26 }
```

## 第十章 配套知识与验证范围

### 10.1 回文半径与在线回文计数

Manacher 的  $odd[i]$  包含中心, 最长奇回文为  $[i - odd[i] + 1, i + odd[i] - 1]$ , 长度  $2odd[i] - 1$ ;  $even[i]$  以  $i$  前间隙为中心, 最长偶回文为  $[i - even[i], i + even[i] - 1]$ , 长度  $2even[i]$ 。所有  $odd/even$  之和是回文的出现总次数, 应以 long long 累加; 这不是不同回文串数量。恢复最长子串时记录对应起点和长度即可。

附件插入分隔符后的半径有包含中心与不含中心两种约定, 不能直接套到本库数组。哨兵版本还依赖字母表: 小写字母的写法不能原样推广到含标记或零字节的输入。本库显式检查边界, 空串和任意字节均合法; 按字节而非 Unicode 字素比较, int 索引要求  $n < 2^{30}$ 。P3805 的 1100 万字符相同、交替和无长度至少二回文的三种闭式输入已通过双模式本地检查。

PAM 的 suffixes 是当前回文节点的非空回文后缀数量, 等于 fail 节点的 suffixes 加一。P5496 必须先按上一个答案解码当前字符, 再 add, 输出  $t[last].suffixes.hits$  仅统计作为最长后缀的命中次数; 完整次数应取 occurrences() 的返回值。该函数复制后再沿 fail 汇总, 不改 hits, 因此重复查询及查询后继续追加均安全。total 为逐位置 suffixes 的总和, 最多  $n(n+1)/2$ ; distinct 才是不同非空回文串数。PAM 固定小写字母, 内部前置 -1 与  $0 \dots 25$  编码不同, 不继承 Manacher 的任意字节契约。

独立逐子串逆序比较、PAM 全短串及两题最大规模驱动均在普通和 ASan/UBSan 下通过。Manacher 的 LC 全部 24 组官方数据双模式本地 checker 接受, 不等于线上 AC。详细接口与记录见仓库 docs/PALINDROME-ATTACHMENT.md。

### 10.2 整数后缀数组的边界与 SAM 次数

SuffixArray 的整数入口要求  $0 \leq a[i] < alphabet$ ; 稀疏大整数先保序离散化。sa/rk/lcp 都从零编号, lcp[k] 比较 sa[k-1] 与 sa[k], 首项为零。附件第二份倍增模板额外要求 LCP 遇分隔符停止, 不能把普通 LCP 不加处理地代入。令 remain[i] 为从 i 到下一个分隔符之前的普通字符数, 则分段前缀长为

$$height[k] = \min\{lcp[k], remain[sa[k-1]], remain[sa[k]]\}.$$

分隔符位置 remain 为零, 连续或重复分隔符同样适用; 保留原 lcp 可避免两种语义混用。两个字符串拼接后, 相同前缀的后缀在 SA 中连续, 所以找最长公共子串只需检查异来源的相邻后缀。输出时转回各自原串的半开区间, 不把分隔符当原字符。

SAM 状态  $u$  代表长度  $(len[link[u]], len[u])$  内具有同一 endpos 集合的子串。真实扩展状态的 occ 种子为一, 克隆为零; 附件存放插入 id 的 endpos 字段不等于完整出现次数。counts 按长度倒序传播, 不能按创建编号倒序, 因为克隆可能后创建。查询返回副本, 不改种子, 所以重复查询后继续 extend 安全。根计数为串长  $n$ , 不能解释成空串的  $n+1$  个出现位置; 非空子串只看非根状态。P3804 仅在  $count[u] > 1$  时取  $count[u] * len[u]$ , 乘积用 long long, 无重复子串则为零。若需要每个前缀对应的状态, 调用者在每次 extend 后保存 last 即可。

独立后缀排序、逐子串 endpos 集合、分隔符截断与完整百万规模驱动双模式通过。LC 后缀数组 50 组、最长公共子串 28 组官方数据双模式本地 checker 接受, 不等于线上 AC 或速度排名。细节见仓库 docs/SUFFIX-ATTACHMENT.md。

### 10.3 广义 SAM 点数与后缀语言

GeneralSAM 的状态按文档编号与结束位置组成的 endpos 集合等价。先 add 全部小写串, 再 build; 根为零, distinct 返回不同非空子串总数, a.size() 包含根。构造后不再 add, 重复 build 安全; 重建另开对象。重复串和空串在库内合法, P6139 则要求输入非空串。

多串 `endpos` 类不能直接称为后缀并集的最小 DFA。例如 `ab`、`b` 的三个非空子串 `a`、`b`、`ab` 具有不同 `endpos` 集合，含根共四点；仅识别后缀并集时，读完 `b` 和 `ab` 的状态可合并，含初态只需三点（省略拒绝汇点）。P6139 官方样例 `aa`、`ab`、`bac`、`caa` 要求不同子串数与点数均为十，符合 `endpos` 口径；把后缀 Trie 按右语言最小化则为七点，不能据题面中的“最小”二字输出七。完整用法直接输出 `a.size()`，不额外做 DFA 合并。

独立枚举文档位置集合与两种右语言计算核对上述区别。完整驱动的 600 组随机参考及排列、百万字符链和克隆密集串、两条各五十万字符的交替串、四十万文档合计百万字符，以及深十五完整二叉 Trie 均通过普通和 ASan/UBSan。这些是本地证据，尚未新增 P6139 线上 AC 或排名。构造与口径详见仓库 `docs/GENERAL-SAM-NOTES.md`。

# 算法索引

ac\_shortest, 6  
AhoCorasick, 4  
  
GeneralSAM, 17  
  
kmp\_match, 1  
  
manacher, 21  
minimum\_rotation, 26  
  
PalindromicTree, 23  
prefix\_function, 1  
prefix\_lcs, 12  
  
square\_counts, 12  
SuffixArray, 8  
SuffixAutomaton, 13  
SuffixLCP, 11  
  
z\_function, 3